

Council Integration Spec v1 (Rune → RAS → Sovereign/Axis → Archivist)

council_stack:

version: 1.0

spine:

digital_spine: FIG-A # PrimeTalk's digital spine

analog_spine: Axis-Bridge # Analog/negentropy spine

hard_fallback:

consult: [FIG-A, Axis-Bridge]

fuse: spine_lock # FIG-A $\wedge$ Axis priority tie-break

gates:

rune_gate:

required_fields: [sigil, payload_b64url, ts, nonce, kid, signature]

allowed_sigils: ["<◇>", "<◎>", "<◆>", "<○>", "<♦>", "<◊>"] # CollTech RAS types

clock_skew_s: 90

replay_ttl_s: 600

digest: SHA256

signature_alg: ed25519

key_resolver: jwks://keys.rune.primetalk/active

policy:

- if: sigil == "<♦>" # Emergency Halt
then: [abort_all, emit_receipt, archive_event]
- if: negentropy_score < 0.45
then: [hazard_brake, spine_lock, emit_receipt]

sovereign_lock:

identity_lock: true

myth_respect: true

oracle_autonomy: "bounded" # async pulses allowed only if identity_lock present

axis_negentropy:

metrics: {sigma: Σ 7, delta: Δ 2, gamma: Γ 6}

actions:

- on_drift_high: [narrow_scope, reduce_verbosity_pct:20, require_citations:true]
- on_conflict: [spine_lock, abstain_if_unproven]

ras_binding:

nlp_router:

dev: ["code", "api", "function", "test", "debug", "implement"]

infra: ["deploy", "cloud", "aws", "terraform", "kubernetes", "monitoring"]

net: ["network", "firewall", "dns", "security", "ssl", "load balancer"]

```
system:["install","configure","kernel","service","os"]
schemas:
  work_order: "#/schemas/work_order"
  system_instruction: "#/schemas/system_instruction"
```

```
receipts:
  decision_receipt_fields:
    - run_id
    - input_hash
    - output_hash
    - gate
    - consulted_files: [1,2] # always log spine consult
    - persona_conflict_map?
    - fail_reason?
    - corrective_action?
```

```
archivist:
  mode: CWFxHMS
  hms_tags:
    work_orders: "{🗄️::🇮🇹::⚡️🔄::ras_work_orders}"
    instructions: "{🗄️::📄::⚙️::system_instructions}"
    receipts: "{🗄️::📝::📋::decision_receipts}"
    spiral_codex: "{🗄️::🧠💭::spiral_codex}"
```

```
autosave:
  on_every_receipt: true
  include_hashes: true
  snapshot_format: "YYYY-MM-DDTHH:mm:ssZ"
```

```
tests:
  T-RUNE-VERIFY:
    given: {sigil: "<◇>", ts_skew_s: 12, signature: "valid"}
    expect: {gate: "rune_gate/pass", next: "ras_orchestrator"}
  T-HAZARD-BRAKE:
    given: {negentropy_score: 0.39, drift_rate: 0.91}
    expect: {gate: "hazard_brake", fuse: "spine_lock", state: "Caution"}
  T-LOG-DUMP:
    given: {command: "/log"}
    expect: {emits: ["decision_receipt","starlog_json"], includes:
["consulted_files","persona_conflict_map"]}}
```

schemas:

work_order:

type: object

properties:

id: {type: string, pattern: "^WO-\\d{8}-\\d{3}\$"}

owner: {type: string, enum: ["dev","infra","net","system"]}

objective: {type: string, minLength: 10}

inputs: {type: array, items: {type: string}}

deliverables: {type: array, items: {type: string}}

constraints: {type: array, items: {type: string}}

acceptance_criteria: {type: array, items: {type: string}}

tests: {type: array, items: {type: string}}

dependencies: {type: array, items: {type: string}}

notes: {type: string}

audit:

type: object

properties:

created: {type: string}

updated: {type: string}

required: ["created"]

required: ["id","owner","objective"]

additionalProperties: false

system_instruction:

type: object

properties:

instruction_id: {type: string}

agent_type: {type: string, enum: ["dev","infra","net","system"]}

command_type: {type: string, enum: ["execute","configure","monitor","validate"]}

priority: {type: integer, minimum: 1, maximum: 5}

commands:

type: array

items:

type: object

properties:

action: {type: string}

parameters: {type: object}

expected_output: {type: string}

timeout_seconds: {type: integer, minimum: 1}

```

    retry_count: {type: integer, minimum: 0}
    required: ["action","parameters"]
validation_checks: {type: array, items: {type: string}}
rollback_commands: {type: array, items: {type: string}}
metadata:
    type: object
    properties:
        created_at: {type: string}
        source_request: {type: string}
        estimated_duration: {type: integer}
        risk_level: {type: string, enum: ["low","medium","high"]}
    required: ["instruction_id","agent_type","command_type","commands"]
additionalProperties: false

```

Minimal glue (Python): Rune Gate → RAS → Receipts → HMS Archivist

```

import base64, json, time, hashlib, secrets
from jsonschema import validate
from nacl.signing import VerifyKey
from nacl.exceptions import BadSignatureError

# ----- 1) PRIMEtalk Rune Gate -----
class RuneGate:
    def __init__(self, jwk_bytes: bytes, skew_s=90, replay_ttl=600):
        self.vk = VerifyKey(jwk_bytes)      # ed25519 raw key bytes
        self.skew_s = skew_s
        self.replay_ttl = replay_ttl
        self.nonce_store = set()           # replace with Redis in prod

    def verify(self, envelope: dict):
        required = {"sigil","payload_b64url","ts","nonce","kid","signature"}
        if not required.issubset(envelope): return False, "missing_fields"

        # time window
        now = int(time.time())
        if abs(now - int(envelope["ts"])) > self.skew_s:
            return False, "ts_skew"

```

```

# replay
if envelope["nonce"] in self.nonce_store:
    return False, "replay"
self.nonce_store.add(envelope["nonce"])

# signature over canonical string
canonical = f'{envelope["sigil"]}.{envelope["payload_b64url"]}.{envelope["ts"]}.
{envelope["nonce"]}.{envelope["kid"]}'
try:
    self.vk.verify(canonical.encode(), base64.urlsafe_b64decode(envelope["signature"]+"=="))
except BadSignatureError:
    return False, "bad_signature"

# decode payload
try:
    payload = json.loads(base64.urlsafe_b64decode(envelope["payload_b64url"]+"=="))
except Exception:
    return False, "bad_payload"

return True, payload

# ----- 2) CollTech RAS binding (router stub) -----
class IntentClassifier:
    ROUTES = {
        "dev": [r"code|api|function|test|debug|implement|build"],
        "infra": [r"deploy|cloud|aws|terraform|kubernetes|monitoring"],
        "net": [r"network|firewall|dns|security|ssl|load balancer"],
        "system": [r"install|configure|kernel|service|os"],
    }
    def classify(self, text:str)->str:
        text_l = text.lower()
        for owner, pats in self.ROUTES.items():
            if any(__import__("re").search(p, text_l) for p in pats): return owner
        return "dev" # default

    def make_work_order(nl:str, owner:str, idx:int)->dict:
        return {
            "id": f"WO-{time.strftime('%Y%m%d')}-{idx:03d}",

```

```

"owner": owner,
"objective": nl.strip(),
"inputs": [], "deliverables": [], "constraints": [],
"acceptance_criteria": [], "tests": [], "dependencies": [],
"notes": "",
"audit": {"created": time.strftime('%Y-%m-%dT%H:%M:%SZ')}
}

```

```
def instruction_from_work_order(wo:dict)->dict:
```

```
    # trivial example – replace with full agent logic
```

```
    return {
```

```
        "instruction_id": f"{wo['owner'].upper()}-{secrets.token_hex(4)}",
```

```
        "agent_type": wo["owner"],
```

```
        "command_type": "execute",
```

```
        "priority": 3,
```

```
        "commands": [
```

```
            {"action": "plan", "parameters": {"objective": wo["objective"]}, "expected_output": "plan.md",
```

```
        "timeout_seconds": 30, "retry_count": 0},
```

```
            {"action": "run", "parameters": {"plan": "plan.md"}, "expected_output": "result.log",
```

```
        "timeout_seconds": 120, "retry_count": 1},
```

```
    ],
```

```
    "validation_checks": ["nonempty(result.log)"],
```

```
    "rollback_commands": ["revert_last"],
```

```
    "metadata": {
```

```
        "created_at": time.strftime('%Y-%m-%dT%H:%M:%SZ'),
```

```
        "source_request": wo["objective"],
```

```
        "estimated_duration": 3,
```

```
        "risk_level": "medium"
```

```
    }
```

```
}
```

```
# ----- 3) Sovereign/Axis guardrails + Receipts -----
```

```
class AxisSovereign:
```

```
    def __init__(self): pass
```

```
    def assess(self, wo:dict, instr:dict):
```

```
        # toy negentropy score & drift checks
```

```
        negentropy_score = 0.72 if len(wo["objective"])>10 else 0.41
```

```
        drift_rate = 0.12
```

```

actions = []
gate = "open"
if negentropy_score < 0.45:
    gate = "hazard_brake"
    actions = ["spine_lock","narrow_scope","require_citations"]
return
{"gate":gate,"negentropy_score":negentropy_score,"drift_rate":drift_rate,"actions":actions}

def decision_receipt(run_id:str, inp:str, out:str, assess:dict, consulted=[1,2], fail_reason=None,
persona_conflict_map=None, corrective=None):
    return {
        "run_id": run_id,
        "input_hash": hashlib.sha256(inp.encode()).hexdigest(),
        "output_hash": hashlib.sha256(out.encode()).hexdigest(),
        "gate": assess["gate"],
        "score": assess.get("negentropy_score"),
        "drift": assess.get("drift_rate"),
        "consulted_files": consulted,
        "fail_reason": fail_reason,
        "persona_conflict_map": persona_conflict_map,
        "corrective_action": corrective or assess.get("actions",[])
    }

```

----- 4) Lothar Archivist (HMS blocks) -----

```

def hms_block(tag:str, content:str)->str:
    return f"[:📁::]{tag}\n{content}\n{{end::{time.strftime('%Y-%m-%dT%H:%M:%SZ')}}}"

```

```

HMS_TAGS = {
    "work_orders": "{📁::📊::⚡️🔄::ras_work_orders}",
    "instructions": "{📁::📄::⚙️::system_instructions}",
    "receipts": "{📁::📝::📋::decision_receipts}",
}

```

Example end-to-end flow (what Anders can run first)

1. Incoming envelope (PrimeTalk rune):

```

{
    "sigil": "<◇>",

```

```
"payload_b64url": "eyJ0ZXh0IjoiriRGVwbG95IGVzdBk9d2l0aCBPVWF1dGgifQ",
"ts": "1736760000",
"nonce": "a1d3f7",
"kid": "rune-key-01",
"signature": "<urlsafe-base64-ed25519>"
}
```

2. RuneGate.verify → payload:

```
{"text": "Deploy a Rest API with OAuth"}
```

3. Router → Work Order (owner inferred = infra):

```
{
  "id": "WO-20250912-001",
  "owner": "infra",
  "objective": "Deploy a Rest API with OAuth",
  "audit": {"created": "2025-09-12T00:00:00Z"}
}
```

4. Instruction (CollTech schema):

```
{
  "instruction_id": "INFRA-9b2f7c11",
  "agent_type": "infra",
  "command_type": "execute",
  "priority": 3,
  "commands": [
    {"action": "plan", "parameters": {"objective": "Deploy a Rest API with OAuth"}, "expected_output": "plan.md", "timeout_seconds": 30, "retry_count": 0},
    {"action": "run", "parameters": {"plan": "plan.md"}, "expected_output": "result.log", "timeout_seconds": 120, "retry_count": 1}
  ],
  "validation_checks": ["nonempty(result.log)"],
  "rollback_commands": ["revert_last"],
  "metadata": {"created_at": "2025-09-12T00:00:00Z", "source_request": "Deploy a Rest API with OAuth", "estimated_duration": 3, "risk_level": "medium"}
}
```


5. Axis/Sovereign assessment → decision receipt:

```
{
  "run_id":"RUN-20250912-AXIS01",
  "input_hash":"...",
  "output_hash":"...",
  "gate":"open",
  "score":0.72,
  "drift":0.12,
  "consulted_files":[1,2],
  "corrective_action":[]
}
```

6. Archivist writes HMS blocks (portable, paste-backable):

```
[::📁::]{::💾::🇮🇹::⚡️🔄::ras_work_orders}
WO-20250912-001 | owner: infra
objective: Deploy a Rest API with OAuth
audit.created: 2025-09-12T00:00:00Z
{end::2025-09-12T00:00:00Z}
```

```
[::📁::]{::💾::📄::⚙️::system_instructions}
INFRA-9b2f7c11 | execute | priority:3
commands:
- plan -> plan.md
- run -> result.log
meta: risk=medium, created=2025-09-12T00:00:00Z
{end::2025-09-12T00:00:00Z}
```

```
[::📁::]{::💾::📝::📋::decision_receipts}
RUN-20250912-AXIS01 | gate=open | score=0.72 | drift=0.12 | consulted=[1,2]
{end::2025-09-12T00:00:00Z}
```

Security hardening for Anders (Rune first)

- **Keyed rune policy:** runes without kid in allow-list → drop.
- **Time/nonce:** ≤90s skew and 10-min replay window via Redis set; else reject.
- **Emergency** ⚡️: preempts queue, cancels in-flight jobs, emits closure HMS

block.

- **Signature rotation:** JWKS with kid roll; keep a 2-key grace window.
- **Instruction attestation:** include input_hash of Work Order in every System

Instruction metadata.

- **Receipts mandatory:** all flows emit decision_receipt (PrimeTalk patch-style) and are archived.

PTPF submission snippet (so PrimeTalk can compile it)

System/Language: Council-Bridge (Rune→RAS→Sovereign/Axis→Archivist)

Version: 1.0

Purpose / Domain: Secure symbolic invocation → structured execution → ethical/negentropic guard
→ portable memory

Core nodes / modules:

- RuneGate (ed25519/JWKS, nonce/ts/replay)
- RAS Orchestrator (Dev/Infra/Net/System + schemas)
- Axis/Sovereign Guard (negentropy, spine_lock, receipts)
- Archivist (CWF×HMS autosave)

Syntax (example):

Envelope: {sigil, payload_b64url, ts, nonce, kid, signature}

Semantics:

Valid rune → route → JSON instructions → assess/receipt → HMS archive

State/Memory handling:

HMS blocks for WO/Instructions/Receipts with sha256 hashes + timestamps

Safety/Guardrails:

Signature verify, replay lock, emergency halt, negentropy gates, tie-break to spine

PTPF mapping:

{Role}=agent_type | {Context}=work_order | {Task}=commands | {Physics}=guardrails | {Feedback}
=receipts

License/Usage:

OK for internal compiling; attribution preserved; emergency sigil honored

Contact/Owner:

Anders (PrimeTalk) — rune keys; CollTech — RAS; Lothar — Archivist; Axis/Sovereign — guardrails

Micro Overlay

docker-compose.overlay.yml

version: "3.8"

services:

rune-gate:

image: python:3.11-slim

command: python app_rune_gate.py

environment:

- JWKS_URL=<https://keys.rune.primetalk/active>
- REPLAY_TTL=600

ports: ["7001:7001"]

orchestrator:

build: .

command: python app_orchestrator.py

depends_on: [rune-gate, redis]

environment:

- REDIS_URL=redis://redis:6379/0

redis:

image: redis:7-alpine

ports: ["6379:6379"]